

Chapter 4. Numerical methods for ordinary differential equations

ARIN7013 Numerical methods in artificial intelligence

The University of Hong Kong



Objective

Numerical methods for ordinary differential equations (stability, convergence analysis, relation between the SGD and Euler method, using DNN to compute ODEs).

Table of Contents

1	Part I (ODEs): Introduction/Preliminaries	4
2	One-step methods	21
■	Euler's method and its relatives: the θ -method	23
■	Error analysis of the θ -method	31
■	General one-step methods	39
■	General explicit one-step methods	42
■	Explicit Runge–Kutta methods (optional)	51
■	Absolute stability of explicit Runge–Kutta methods (optional)	65
3	From ODE to DL	72
■	Residual Neural Networks	74
■	Neural ODEs	85
■	Applications	99
4	Summary	106

4.1 Part I (ODEs): Introduction/Preliminaries

What are Ordinary Differential Equations (ODEs)?

An ODE is an equation for a fct $y = y(x)$ containing derivatives of y , e.g.,

- $y'(x) = 3y(x)$,
- $(y(x) + y'(x) + y''''(x))^7 = y''(x) - y''''(x) + x^3$.

Explicit ODE of order n :

$$y^{(n)}(x) = f(x, y(x), y'(x), \dots, y^{(n-1)}(x)).$$

Implicit ODE of order n :

$$F(x, y(x), y'(x), \dots, y^{(n)}(x)) = 0.$$

In this section, we focus on **explicit first-order ODEs**, i.e.,

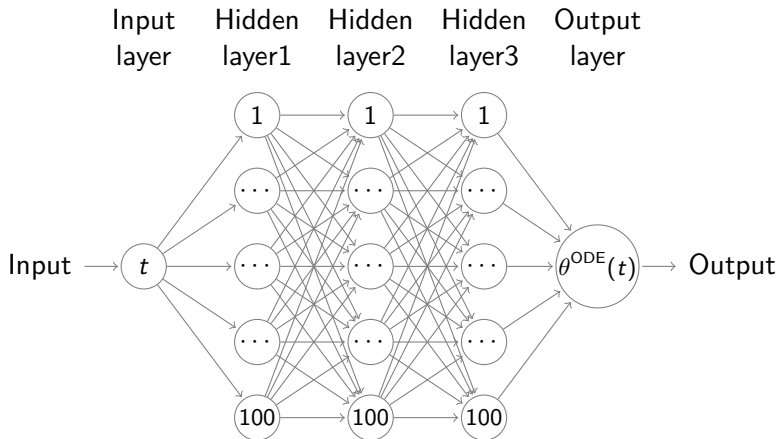
$$y'(x) = f(x, y(x)).$$

Motivation: neural ODE

Neural ODEs parameterize the derivative of the hidden state using a neural network.

$$\partial_t u(t) = f(u(t), \theta^{\text{ODE}}(t)), t \in (0, T], u(0) = u_0;$$

Here, f is a neural network!



Neural ODEs parameterize the derivative of the hidden state using a neural network.

$$\partial_t u(t) = f(u(t), \theta^{\text{ODE}}(t)), t \in (0, T], u(0) = u_0; \quad (1)$$

Here, f is a neural network!

- Given the terminal time T , one can calculate $u(T; \theta^{\text{ODE}}(t))$, and thus the hypothesis space is

$$\mathcal{H} := \{u(T; \theta^{\text{ODE}}(t)) : \text{for any } \theta^{\text{ODE}}(t)\}$$

- Provided with the training data: $\{(y^{(i)}, c^{(i)})_{i=1}^N\}$, one can find a good approximate function to complete a certain task.

Question

- Is there a unique solution to Problem (1)?
- If there is, then how to calculate $u(T; \theta^{\text{ODE}}(t))$ for given parameter $\theta^{\text{ODE}}(t)$?

Initial-value Problems (IVPs)

Ex.: $y'(x) = 3y(x)$ has infinitely many solns: $y(x) = c e^{3x}$, $c \in \mathbb{R}$.

$\implies$ To get a unique solution, we need to specify y at some value $x_0 \in \mathbb{R}$.

$$y'(x) = 3y(x), \quad y(x_0) = y_0 \quad \implies \quad y(x) = y_0 e^{3(x-x_0)}.$$

$y(x_0) = y_0$ is called an **initial condition (i.c.)**. Problems of the form

$$y'(x) = f(x, y(x)), \quad y(x_0) = y_0 \quad (2)$$

are called **Initial-value Problems (IVPs)**.

Note: **Not all IVPs have a unique solution.** E.g.,

$$y'(x) = [y(x)]^{\frac{2}{3}}, \quad y(0) = 0$$

has multiple solutions: $y \equiv 0$ and $y(x) = \frac{1}{27}x^3$. (Here, $f(x, z) := z^{\frac{2}{3}}$)

$\implies f(\cdot, \cdot)$ continuous not sufficient for uniqueness. So what is sufficient?

Rk: f continuous $\implies$ (2) has at least one solution (Peano Existence Thm)

Picard's Theorem for IVPs $y'(x) = f(x, y(x))$, $y(x_0) = y_0$

Theorem (Picard's Theorem)

Suppose $f(\cdot, \cdot)$ is continuous in a region $U \subseteq \mathbb{R}^2$ containing the rectangle $R := [x_0, X_M] \times [y_0 - Y_M, y_0 + Y_M]$, where $X_M > x_0$, $Y_M > 0$, *suppose*

$$\exists L > 0 : \quad |f(x, z) - f(x, \tilde{z})| \leq L|z - \tilde{z}| \quad \forall (x, z), (x, \tilde{z}) \in R, \quad (3)$$

and with $M := \max_{(x,z) \in R} |f(x, z)|$, suppose $M(X_M - x_0) \leq Y_M$. Then, $\exists$ a unique continuously differentiable fct $y : [x_0, X_M] \rightarrow \mathbb{R}$ s.t.

$$y'(x) = f(x, y(x)) \quad \text{for } x \in (x_0, X_M), \quad y(x_0) = y_0.$$

(3) is called **Lipschitz condition** (in 2nd argument of f), L is called **Lipschitz constant**.

Proof: See any introductory ODE course or book. Idea: **Picard iteration**:

$$y_0(x) \equiv y_0, \quad y_n(x) := y_0 + \int_{x_0}^x f(t, y_{n-1}(t)) dt, \quad n \in \mathbb{N}.$$

Then, $y_n \rightarrow y$ in $C([x_0, X_M])$ and y solves the IVP. □

Theorem (Picard's Theorem)

Suppose $f(\cdot, \cdot)$ is continuous in a region $U \subseteq \mathbb{R}^2$ containing the rectangle $R := [x_0, X_M] \times [y_0 - Y_M, y_0 + Y_M]$, where $X_M > x_0$, $Y_M > 0$, suppose

$$\exists L > 0 : \quad |f(x, z) - f(x, \tilde{z})| \leq L|z - \tilde{z}| \quad \forall (x, z), (x, \tilde{z}) \in R,$$

and with $M := \max_{(x,z) \in R} |f(x, z)|$, suppose $M(X_M - x_0) \leq Y_M$. Then, $\exists$ a unique continuously differentiable fct $y : [x_0, X_M] \rightarrow \mathbb{R}$ s.t.

$$y'(x) = f(x, y(x)) \quad \text{for } x \in (x_0, X_M), \quad y(x_0) = y_0.$$

Observation: the graph of y lies in R , i.e., $(x, y(x)) \in R \quad \forall x \in [x_0, X_M]$.

Pf of observation: Suppose this were not true. Then, by continuity of y ,

$$\exists x_* \in (x_0, X_M) : \quad |y(x_*) - y_0| = Y_M, \quad |y(x) - y_0| < Y_M \quad \forall x \in [x_0, x_*].$$

$$\begin{aligned} \Rightarrow |y(x_*) - y_0| &= \left| \int_{x_0}^{x_*} y'(x) dx \right| \leq \int_{x_0}^{x_*} |y'(x)| dx \\ &= \int_{x_0}^{x_*} |f(x, y(x))| dx \leq M(x_* - x_0) < Y_M. \quad \text{!} \quad \square \end{aligned}$$

IVPs for systems of ODEs

IVPs for systems of m ODEs:

$$\begin{cases} y_1'(x) = f_1(x, y_1(x), y_2(x), \dots, y_m(x)), & y_1(x_0) = y_{0,1}, \\ y_2'(x) = f_2(x, y_1(x), y_2(x), \dots, y_m(x)), & y_2(x_0) = y_{0,2}, \\ \vdots \\ y_m'(x) = f_m(x, y_1(x), y_2(x), \dots, y_m(x)), & y_m(x_0) = y_{0,m}, \end{cases}$$

or equivalently,

$$\mathbf{y}'(x) = \mathbf{f}(x, \mathbf{y}(x)), \quad \mathbf{y}(x_0) = \mathbf{y}_0$$

with given

$$x_0 \in \mathbb{R}, \quad \mathbf{y}_0 = \begin{pmatrix} y_{0,1} \\ \vdots \\ y_{0,m} \end{pmatrix} \in \mathbb{R}^m, \quad \mathbf{f} = \begin{pmatrix} f_1 \\ \vdots \\ f_m \end{pmatrix} : [x_0, X_M] \times \mathbb{R}^m \rightarrow \mathbb{R}^m,$$

and we are seeking a soln $\mathbf{y} = (y_1, \dots, y_m)^T : [x_0, X_M] \rightarrow \mathbb{R}^m$.

Picard's Thm for systems $\mathbf{y}'(x) = \mathbf{f}(x, \mathbf{y}(x))$, $\mathbf{y}(x_0) = \mathbf{y}_0$

Introduce the Euclidean norm $\|\cdot\| : \mathbb{R}^m \rightarrow [0, \infty)$ on $\mathbb{R}^m$ by

$$\|\mathbf{u}\| := \sqrt{\sum_{i=1}^m |u_i|^2}, \quad \text{for } \mathbf{u} = (u_1, \dots, u_m)^T \in \mathbb{R}^m.$$

Theorem (Picard's Theorem (version for systems))

Suppose that $\mathbf{f}(\cdot, \cdot)$ is a continuous in a region $U \subseteq \mathbb{R}^{1+m}$ containing

$$R = \{(x, \mathbf{z}) \in \mathbb{R} \times \mathbb{R}^m : x \in [x_0, X_M], \quad \|\mathbf{z} - \mathbf{y}_0\| \leq Y_M\},$$

where $X_M > x_0$, $Y_M > 0$, suppose that

$$\exists L > 0 : \quad \|\mathbf{f}(x, \mathbf{z}) - \mathbf{f}(x, \tilde{\mathbf{z}})\| \leq L \|\mathbf{z} - \tilde{\mathbf{z}}\| \quad \forall (x, \mathbf{z}), (x, \tilde{\mathbf{z}}) \in R,$$

and with $M := \max_{(x, \mathbf{z}) \in R} \|\mathbf{f}(x, \mathbf{z})\|$, suppose $M(X_M - x_0) \leq Y_M$. Then, $\exists$ a unique continuously differentiable fct $\mathbf{y} : [x_0, X_M] \rightarrow \mathbb{R}^m$ s.t.

$$\mathbf{y}'(x) = \mathbf{f}(x, \mathbf{y}(x)), \quad \mathbf{y}(x_0) = \mathbf{y}_0.$$

A sufficient condition guaranteeing the Lipschitz property

Recall Lipschitz condition for IVPs:

$$\exists L > 0 : \quad |f(x, z) - f(x, \tilde{z})| \leq L|z - \tilde{z}| \quad \forall (x, z), (x, \tilde{z}) \in R. \quad (4)$$

This is automatically satisfied if f is cts on R , differentiable in $\text{int}(R)$, and

$$\exists C > 0 : \quad |f_z(x, z)| := \left| \frac{\partial f}{\partial z}(x, z) \right| \leq C \quad \forall (x, z) \in \text{int}(R). \quad (5)$$

Indeed, suppose that (5) holds. By the Mean-Value Thm, for any $(x, z) \in R$ we have $f(x, z) - f(x, \tilde{z}) = f_z(x, \xi)(z - \tilde{z})$ for some ξ between z and $\tilde{z}$. So, we obtain (4) with $L := C$.

Recall Lipschitz condition for IVPs for systems:

$$\exists L > 0 : \quad \|\mathbf{f}(x, \mathbf{z}) - \mathbf{f}(x, \tilde{\mathbf{z}})\| \leq L\|\mathbf{z} - \tilde{\mathbf{z}}\| \quad \forall (x, \mathbf{z}), (x, \tilde{\mathbf{z}}) \in R.$$

This is automatically satisfied with $L = C$ if $\mathbf{f}$ cts on R , diff. in $\text{int}(R)$, and

$$\exists C > 0 : \quad \left\| \frac{\partial \mathbf{f}}{\partial \mathbf{z}}(x, \mathbf{z}) \right\| \leq C \quad \forall (x, \mathbf{z}) \in \text{int}(R), \quad (6)$$

where $\|\cdot\|$ in (6) is the matrix norm induced by the Euclidean vector norm.

Warning: The converse is not necessarily true!

E.g., consider

$$f : \mathbb{R}^2 \rightarrow \mathbb{R}, \quad f(x, z) := |z|$$

and $R := [x_0, X_M] \times [y_0 - Y_M, y_0 + Y_M]$ with $x_0 := y_0 := 0$ and any chosen $X_M, Y_M > 0$. Then,

- f satisfies the Lipschitz condition with $L = 1$:

$$|f(x, z) - f(x, \tilde{z})| = ||z| - |\tilde{z}|| \leq |z - \tilde{z}| \quad \forall (x, z), (x, \tilde{z}) \in R,$$

- but f is not differentiable in $\text{int}(R)$ (as $z \mapsto |z|$ is not diff. at $z = 0$).

Definition (Stability)

Consider an IVP for a system of ODEs $\mathbf{y}'(x) = \mathbf{f}(x, \mathbf{y}(x))$, $\mathbf{y}(x_0) = \mathbf{y}_0$.

- (i) A solution $\mathbf{y} = \mathbf{v}(x)$ is called **stable** on $[x_0, X_M]$ if $\forall \varepsilon > 0 \exists \delta > 0$ s.t. for all $\mathbf{z} \in \mathbb{R}^m$ satisfying $\|\mathbf{y}_0 - \mathbf{z}\| < \delta$ the solution $\mathbf{w}$ to

$$\mathbf{w}'(x) = \mathbf{f}(x, \mathbf{w}(x)), \quad \mathbf{w}(x_0) = \mathbf{z}$$

is defined in $[x_0, X_M]$ and satisfies $\|\mathbf{v}(x) - \mathbf{w}(x)\| < \varepsilon \forall x \in [x_0, X_M]$.

- (ii) A soln $\mathbf{y} = \mathbf{v}(x)$ which is stable on $[x_0, \infty)$ (i.e. stable on $[x_0, X_M]$ for each X_M and with δ independent of X_M) is called **stable in the sense of Lyapunov**.
- (iii) If in addition to (ii) there holds

$$\lim_{x \rightarrow \infty} \|\mathbf{v}(x) - \mathbf{w}(x)\| = 0,$$

then the solution $\mathbf{y} = \mathbf{v}(x)$ is called **asymptotically stable**.

Stability: Example

Recall defn: A solution $\mathbf{y} = \mathbf{v}(x)$ is called **stable** on $[x_0, X_M]$ if $\forall \varepsilon > 0 \exists \delta > 0$ s.t. for all $\mathbf{z} \in \mathbb{R}^m$ satisfying $\|\mathbf{y}_0 - \mathbf{z}\| < \delta$ the solution $\mathbf{w}$ to $\mathbf{w}'(x) = \mathbf{f}(x, \mathbf{w}(x))$, $\mathbf{w}(x_0) = \mathbf{z}$ is defined in $[x_0, X_M]$ and satisfies $\|\mathbf{v}(x) - \mathbf{w}(x)\| < \varepsilon \forall x \in [x_0, X_M]$.

For some fixed $\lambda \in \mathbb{R}$, consider the IVP

$$y'(x) = \lambda y(x), \quad y(0) = 1$$

with unique soln $y = v(x)$ with $v(x) := e^{\lambda x}$.

Question: For what values of λ is the soln $y = v(x)$ stable on $[0, \infty)$ (i.e., stable on $[0, X_M]$ for any $X_M > 0$ and δ independent of X_M)?

For $z \in \mathbb{R}$, the problem

$$w'(x) = \lambda w(x), \quad w(0) = z$$

has the unique soln $w(x) := ze^{\lambda x}$. Note that $|v(x) - w(x)| = |1 - z|e^{\lambda x}$.

- Case $\lambda \leq 0$: Then, $|v(x) - w(x)| \leq |1 - z| \forall x \in [0, \infty)$.

$\implies y = v(x)$ stable on $[0, \infty)$ when $\lambda \leq 0$.

- Case $\lambda > 0$: Note $\max_{x \in [0, X_M]} |v(x) - w(x)| = |1 - z|e^{\lambda X_M}$.

$\implies y = v(x)$ is unstable on $[0, \infty)$ when $\lambda > 0$.

Main result on stability

Theorem (Stability under assumptions of Picard's Thm)

Under assumptions of Picard's Thm, the unique solution $\mathbf{y} = \mathbf{v}(x)$ to $\mathbf{y}'(x) = \mathbf{f}(x, \mathbf{y}(x))$, $\mathbf{y}(x_0) = \mathbf{y}_0$ is stable on $[x_0, X_M]$.

Proof: Let $\mathbf{w}$ be the soln to $\mathbf{w}'(x) = \mathbf{f}(x, \mathbf{w}(x))$, $\mathbf{w}(x_0) = \mathbf{z}$. Then, we have $\mathbf{v}(x) = \mathbf{v}(x_0) + \int_{x_0}^x \mathbf{f}(t, \mathbf{v}(t))dt$ and $\mathbf{w}(x) = \mathbf{z} + \int_{x_0}^x \mathbf{f}(t, \mathbf{w}(t))dt$.

$$\begin{aligned}\|\mathbf{v}(x) - \mathbf{w}(x)\| &\leq \|\mathbf{v}(x_0) - \mathbf{z}\| + \left\| \int_{x_0}^x (\mathbf{f}(t, \mathbf{v}(t)) - \mathbf{f}(t, \mathbf{w}(t)))dt \right\| \\ &\leq \|\mathbf{v}(x_0) - \mathbf{z}\| + \int_{x_0}^x \|\mathbf{f}(t, \mathbf{v}(t)) - \mathbf{f}(t, \mathbf{w}(t))\|dt \\ &\leq \|\mathbf{v}(x_0) - \mathbf{z}\| + L \int_{x_0}^x \|\mathbf{v}(t) - \mathbf{w}(t)\|dt.\end{aligned}$$

Gronwall Lemma: $A(x) \leq a + L \int_{x_0}^x A(t)dt \forall x \implies A(x) \leq ae^{L(x-x_0)} \forall x.$

Proof of Gronwall Lemma: Write $I := [x_0, X_M]$. We need to show that

$$A(x) \leq a + L \int_{x_0}^x A(t) dt \quad \forall x \in I \quad \implies \quad A(x) \leq a e^{L(x-x_0)} \quad \forall x \in I.$$

Multiplying the inequality $-a - L \int_{x_0}^x A(t) + A(x) \leq 0$ by e^{-Lx} :

$$\frac{d}{dx} \left[\frac{a}{L} e^{-Lx} + e^{-Lx} \int_{x_0}^x A(t) dt \right] \leq 0 \quad \forall x \in I.$$

So, the function in the brackets is non-increasing. Thus,

$$\frac{a}{L} e^{-Lx} + e^{-Lx} \int_{x_0}^x A(t) dt \leq \frac{a}{L} e^{-Lx_0} \quad \forall x \in I.$$

Multiply by Le^{Lx} :

$$a + L \int_{x_0}^x A(t) dt \leq a e^{L(x-x_0)} \quad \forall x \in I.$$

It follows that $A(x) \leq a + L \int_{x_0}^x A(t) dt \leq a e^{L(x-x_0)} \quad \forall x \in I.$ □

End of "Section 1: Introduction/Preliminaries".

Chapter 4. Numerical methods for ordinary differential equations

ARIN7013 Numerical methods in artificial intelligence

The University of Hong Kong



4.2 One-step methods

The problem and the standing assumption

We consider the IVP

$$y'(x) = f(x, y(x)), \quad y(x_0) = y_0.$$

We suppose throughout that f satisfies the conditions of Picard's Theorem on the rectangle R and that the IVP has a unique solution defined on $[x_0, X_M]$, $-\infty < x_0 < X_M < \infty$.

1 Euler's method and its relatives: the θ -method

The simplest one-step method: Euler's method

Problem: approximate the soln $y : [x_0, X_M] \rightarrow \mathbb{R}$ to the IVP

$$y'(x) = f(x, y(x)) \quad \text{for } x \in (x_0, X_M), \quad y(x_0) = y_0.$$

For $N \in \mathbb{N}$, we divide the interval $[x_0, X_M]$ by the $N + 1$ **mesh-points**

$$x_0, \quad x_1 = x_0 + h, \quad x_2 = x_0 + 2h, \quad \dots, \quad x_N = x_0 + Nh = X_M,$$

where $h := \frac{X_M - x_0}{N} > 0$ is the so-called **step size**. For $n \in \{0, \dots, N\}$, we want to find an approximation y_n to $y(x_n)$. $n = 0$ is already done by i.c.!

Idea:
$$y(x_{n+1}) = y(x_n) + \int_{x_n}^{x_{n+1}} y'(x) dx = y(x_n) + \int_{x_n}^{x_{n+1}} f(x, y(x)) dx.$$

Rectangle rule:
$$\int_a^b g(x) dx \approx (b - a)g(a)$$

$\implies y(x_{n+1}) \approx y(x_n) + hf(x_n, y(x_n)).$ **(Explicit) Euler method:**

$$y_{n+1} = y_n + hf(x_n, y_n), \quad n \in \{0, \dots, N - 1\}.$$

Generalization: the θ -method

Idea: As before,

$$y(x_{n+1}) = y(x_n) + \int_{x_n}^{x_{n+1}} f(x, y(x)) dx.$$

Now, instead of rectangle rule, let us use the integration rule

$$\int_a^b g(x) dx \approx (b-a) [(1-\theta)g(a) + \theta g(b)], \quad \theta \in [0, 1].$$

We find that

$$y(x_{n+1}) \approx y(x_n) + h [(1-\theta)f(x_n, y(x_n)) + \theta f(x_{n+1}, y(x_{n+1}))].$$

The θ -method with $\theta \in [0, 1]$:

$$y_{n+1} = y_n + h [(1-\theta)f(x_n, y_n) + \theta f(x_{n+1}, y_{n+1})], \quad n \in \{0, \dots, N-1\}.$$

To compute y_{n+1} , only need the previous value $y_n \implies$ **One-step method**

The choices $\theta = 0$, $\theta = 1$, and $\theta = \frac{1}{2}$

Recall the θ -method with $\theta \in [0, 1]$:

$$y_{n+1} = y_n + h[(1 - \theta)f(x_n, y_n) + \theta f(x_{n+1}, y_{n+1})], \quad n \in \{0, \dots, N - 1\}.$$

■ The choice $\theta = 0$: **Explicit Euler method**

$$y_{n+1} = y_n + hf(x_n, y_n), \quad n \in \{0, \dots, N - 1\}.$$

■ The choice $\theta = 1$: **Implicit Euler method**

$$y_{n+1} = y_n + hf(x_{n+1}, y_{n+1}), \quad n \in \{0, \dots, N - 1\}.$$

■ The choice $\theta = \frac{1}{2}$: **Trapezium rule method**

$$y_{n+1} = y_n + h \frac{f(x_n, y_n) + f(x_{n+1}, y_{n+1})}{2}, \quad n \in \{0, \dots, N - 1\}.$$

Rk: If instead of averaging $f(x_n, y_n), f(x_{n+1}, y_{n+1})$, we evaluate f at the average of x_n, x_{n+1} and y_n, y_{n+1} we obtain the **implicit midpoint rule**:

$$y_{n+1} = y_n + hf\left(\frac{x_n + x_{n+1}}{2}, \frac{y_n + y_{n+1}}{2}\right), \quad n \in \{0, \dots, N - 1\}.$$

So, which $\theta \in [0, 1]$ is best? An experiment

Consider the IVP

$$y'(x) = x - [y(x)]^2 \quad \text{for } x \in (0, 0.4), \quad y(0) = 0.$$

(Note: here, $f : \mathbb{R}^2 \rightarrow \mathbb{R}$, $f(x, z) := x - z^2$.) We take the mesh points

$$x_0 := 0, \quad x_1 := 0.1, \quad x_2 := 0.2, \quad x_3 := 0.3, \quad x_4 := 0.4 \quad (h := 0.1).$$

Choice $\theta = 0$ (explicit Euler): $y_{n+1} = y_n + hf(x_n, y_n)$

$$y_0 = y(x_0) = 0,$$

$$y_1 = y_0 + hf(x_0, y_0) = 0 + 0.1 \cdot f(0, 0) = 0,$$

$$y_2 = y_1 + hf(x_1, y_1) = 0 + 0.1 \cdot f(0.1, 0) = 0.01,$$

$$y_3 = y_2 + hf(x_2, y_2) = 0.01 + 0.1 \cdot f(0.2, 0.01) = 0.02999,$$

$$y_4 = y_3 + hf(x_3, y_3) = 0.02999 + 0.1 \cdot f(0.3, 0.02999) \approx 0.05990.$$

Values of the true solution y at $x_1, \dots, x_4$ (to 5 digits after comma):

$$y(x_1) \approx 0.00500, \quad y(x_2) \approx 0.01998, \quad y(x_3) \approx 0.04488, \quad y(x_4) \approx 0.07949.$$

We see that the largest error $|e_n| := |y(x_n) - y_n|$ is $|e_4| \approx 1.96 \cdot 10^{-2}$.

So, which $\theta \in [0, 1]$ is best? An experiment

Consider the IVP

$$y'(x) = x - [y(x)]^2 \quad \text{for } x \in (0, 0.4), \quad y(0) = 0.$$

(Note: here, $f : \mathbb{R}^2 \rightarrow \mathbb{R}$, $f(x, z) := x - z^2$.) We take the mesh points

$$x_0 := 0, \quad x_1 := 0.1, \quad x_2 := 0.2, \quad x_3 := 0.3, \quad x_4 := 0.4 \quad (h := 0.1).$$

Choice $\theta = 1$ (implicit Euler): $y_{n+1} = y_n + hf(x_{n+1}, y_{n+1})$

$$y_0 = y(x_0) = 0,$$

$$y_1 = y_0 + hf(x_1, y_1) = 0.1(0.1 - y_1^2) \implies y_1 \approx 0.00999,$$

$$y_2 = y_1 + hf(x_2, y_2) \implies y_2 \approx 0.02990,$$

$$y_3 = y_2 + hf(x_3, y_3) \implies y_3 \approx 0.05955,$$

$$y_4 = y_3 + hf(x_4, y_4) \implies y_4 \approx 0.09857.$$

Values of the true solution y at $x_1, \dots, x_4$ (to 5 digits after comma):

$$y(x_1) \approx 0.00500, \quad y(x_2) \approx 0.01998, \quad y(x_3) \approx 0.04488, \quad y(x_4) \approx 0.07949.$$

We see that the largest error $|e_n| := |y(x_n) - y_n|$ is $|e_4| \approx 1.91 \cdot 10^{-2}$.

So, which $\theta \in [0, 1]$ is best? An experiment

Consider the IVP

$$y'(x) = x - [y(x)]^2 \quad \text{for } x \in (0, 0.4), \quad y(0) = 0.$$

(Note: here, $f : \mathbb{R}^2 \rightarrow \mathbb{R}$, $f(x, z) := x - z^2$.) We take the mesh points

$$x_0 := 0, \quad x_1 := 0.1, \quad x_2 := 0.2, \quad x_3 := 0.3, \quad x_4 := 0.4 \quad (h := 0.1).$$

Choice $\theta = \frac{1}{2}$ (trap. rule method): $y_{n+1} = y_n + \frac{h}{2}(f(x_n, y_n) + f(x_{n+1}, y_{n+1}))$.

$$y_0 = y(x_0) = 0,$$

$$y_1 = y_0 + \frac{h}{2}(f(x_0, y_0) + f(x_1, y_1)) = 0.05(0.1 - y_1^2) \implies y_1 \approx 0.00500,$$

$$y_2 = y_1 + \frac{h}{2}(f(x_1, y_1) + f(x_2, y_2)) \implies y_2 \approx 0.01998,$$

$$y_3 = y_2 + \frac{h}{2}(f(x_2, y_2) + f(x_3, y_3)) \implies y_3 \approx 0.04486,$$

$$y_4 = y_3 + \frac{h}{2}(f(x_3, y_3) + f(x_4, y_4)) \implies y_4 \approx 0.07944.$$

Values of the true solution y at $x_1, \dots, x_4$ (to 5 digits after comma):

$$y(x_1) \approx 0.00500, \quad y(x_2) \approx 0.01998, \quad y(x_3) \approx 0.04488, \quad y(x_4) \approx 0.07949.$$

So, which $\theta \in [0, 1]$ is best? An experiment

$\Rightarrow \theta = \frac{1}{2}$ seems to be much better than $\theta = 0$ or $\theta = 1$. Why?

$\Rightarrow$ Will be clear after the next section.

Remark: The true solution in this example is not available in closed form. We have used Picard iteration

$$y_0(x) \equiv 0, \quad y_k(x) = \int_0^x (t - [y_{k-1}(t)]^2) dt, \quad k \in \mathbb{N}.$$

to get a very fine approximation of the soln, which acts as true soln.

$$y_0(x) \equiv 0 \quad y_1(x) = \frac{1}{2}x^2, \quad y_2(x) = \frac{1}{2}x^2 - \frac{1}{20}x^5, \quad \dots$$

By induction, one shows that

$$y(x) = \frac{1}{2}x^2 - \frac{1}{20}x^5 + \frac{1}{160}x^8 - \frac{1}{4400}x^{11} + O(x^{14}).$$

2. Error analysis of the θ -method

The goal of this section

Consider the θ -method for $\theta \in [0, 1]$:

$$y_{n+1} = y_n + h [(1 - \theta)f(x_n, y_n) + \theta f(x_{n+1}, y_{n+1})], \quad n \in \{0, \dots, N-1\}.$$

We define the **global error** e_n by

$$e_n := y(x_n) - y_n \quad \text{for } n \in \{0, \dots, N\}.$$

We investigate the decay of the global error for the θ -method with respect to the reduction of the mesh size h .

For simplicity, we consider $\theta = 0$ (i.e., explicit Euler) and then only state the result for general $\theta \in [0, 1]$ in the end (exercise).

Error analysis for explicit Euler. 1) Consistency error

Explicit Euler method:

$$y_{n+1} = y_n + hf(x_n, y_n), \quad n \in \{0, 1, \dots, N-1\}, \quad y_0 = y(x_0).$$

Introduce the **consistency error** (or **truncation error**):

$$T_n = \frac{y(x_{n+1}) - y(x_n)}{h} - f(x_n, y(x_n)), \quad n \in \{0, 1, \dots, N-1\},$$

By $f(x_n, y(x_n)) = y'(x_n)$ and Taylor's Theorem, $\exists \xi_n \in (x_n, x_{n+1})$ s.t.

$$|T_n| = \frac{|y(x_{n+1}) - y(x_n) - hy'(x_n)|}{h} = \frac{\frac{1}{2}h^2|y''(\xi_n)|}{h} \leq \frac{h}{2}M_2$$

with $M_2 := \max_{x \in [x_0, X_M]} |y''(x)|$, where we have assumed f is sufficiently smooth so that $y''(x) = \frac{d}{dx}[f(x, y(x))]$ exists and is bounded on $[x_0, X_M]$.

Error analysis for explicit Euler. 2) Compare e_{n+1} to e_n

By definition of explicit Euler:

$$0 = \frac{y_{n+1} - y_n}{h} - f(x_n, y_n). \quad (7)$$

By definition of consistency error:

$$T_n = \frac{y(x_{n+1}) - y(x_n)}{h} - f(x_n, y(x_n)). \quad (8)$$

Recall $e_n = y(x_n) - y_n$. Subtract (7) from (8):

$$T_n = \frac{e_{n+1} - e_n}{h} - [f(x_n, y(x_n)) - f(x_n, y_n)],$$

or equivalently,

$$e_{n+1} = e_n + h[f(x_n, y(x_n)) - f(x_n, y_n)] + hT_n.$$

Assuming that $|y_n - y_0| \leq Y_M$, from the Lipschitz condition we get

$$|e_{n+1}| \leq |e_n| + h|f(x_n, y(x_n)) - f(x_n, y_n)| + h|T_n| \leq (1 + hL)|e_n| + h|T_n|.$$

Now, let $T := \max\{|T_0|, |T_1|, \dots, |T_{N-1}|\}$; then,

$$|e_{n+1}| \leq (1 + hL)|e_n| + hT \quad \forall n \in \{0, \dots, N-1\}.$$

Error analysis for explicit Euler. 3) Compare e_n to e_0

Using $|e_{n+1}| \leq (1 + hL)|e_n| + hT$ from Step 2, we find

$$\begin{aligned}|e_n| &\leq (1 + hL)|e_{n-1}| + hT \\ &\leq (1 + hL)((1 + hL)|e_{n-2}| + hT) + hT \\ &\vdots \\ &\leq (1 + hL)^n |e_0| + \frac{T}{L} [(1 + hL)^n - 1].\end{aligned}$$

Noting that $1 + x \leq e^x \forall x \in \mathbb{R}$, and $nh = x_n - x_0$, we have

$$|e_n| \leq e^{nhL} |e_0| + \frac{T}{L} [e^{nhL} - 1] = e^{L(x_n - x_0)} |e_0| + \frac{T}{L} [e^{L(x_n - x_0)} - 1].$$

Using $T \leq \frac{h}{2} M_2$ from Step 1 gives

$$|e_n| \leq e^{L(x_n - x_0)} |e_0| + h \frac{M_2}{2L} [e^{L(x_n - x_0)} - 1] \quad \forall n \in \{0, \dots, N\}.$$

Error analysis for explicit Euler. 4) Conclude

We have found in Step 3 that

$$|e_n| \leq e^{L(x_n - x_0)} |e_0| + h \frac{M_2}{2L} \left[e^{L(x_n - x_0)} - 1 \right] \quad \forall n \in \{0, \dots, N\}.$$

Observation: $e_0 = y(x_0) - y_0 = 0$. (at least in theory; in practice, y_0 is the nearest floating point number to $y(x_0)$). Thus,

$$|e_n| \leq h \frac{M_2}{2L} \left[e^{L(x_n - x_0)} - 1 \right] \leq h \frac{M_2}{2L} \left[e^{L(X_M - x_0)} - 1 \right] \quad \forall n \in \{0, \dots, N\}.$$

We conclude that

$$\max_{n \in \{0, \dots, N\}} |e_n| = \mathcal{O}(h)$$

as $h \searrow 0$.

Error analysis for the θ -method with $\theta \in [0, 1]$

Exercise.

For $\theta \neq \frac{1}{2}$, one finds that

$$\max_{n \in \{0, \dots, N\}} |e_n| = \mathcal{O}(h),$$

and for $\theta = \frac{1}{2}$, one find that

$$\max_{n \in \{0, \dots, N\}} |e_n| = \mathcal{O}(h^2).$$

$\implies \theta = \frac{1}{2}$ “best” value of θ regarding speed of convergence to true soln.

So, why not always use $\theta = \frac{1}{2}$?

While the trapezium rule method leads to more accurate approximations than the explicit Euler method, it is **less convenient from the computational point of view** because it requires the solution of implicit equations at each mesh point x_{n+1} to compute y_{n+1} .

Attractive compromise: use explicit Euler to compute an initial crude approximation to $y(x_{n+1})$ and then use this value within the trapezium rule to obtain a more accurate approximation for $y(x_{n+1})$:

$$y_{n+1} = y_n + \frac{h}{2}[f(x_n, y_n) + f(x_{n+1}, y_n + hf(x_n, y_n))], \quad n \in \{0, \dots, N-1\}.$$

This is called the **improved Euler method**.

3. General one-step methods

Formal definition of a one-step method

Definition (One-step method)

A **one-step method** is a fct Ψ that takes $(\xi, \eta; h) \in \mathbb{R} \times \mathbb{R} \times (0, \infty)$ and a fct $f(\cdot, \cdot)$, and computes an approximation $\Psi(\xi, \eta; h, f) \in \mathbb{R}$ of $y(\xi + h)$, which is the soln at $x = \xi + h$ of the IVP

$$y'(x) = f(x, y(x)), \quad y(\xi) = \eta. \quad (9)$$

We assume that (9) has a unique soln, and therefore $y(\xi + h)$ exists. Here, h may need to be sufficiently small for Ψ to be well-defined.

- explicit Euler: $\Psi(\xi, \eta; h, f) = \eta + hf(\xi, \eta)$.
- implicit Euler: Ψ is defined implicitly, by

$$\Psi(\xi, \eta; h, f) = \eta + hf(\xi + h, \Psi(\xi, \eta; h, f)).$$

If f satisfies a **global Lipschitz condition** (in 2nd arg.), i.e., $\exists L > 0$:

$$|f(x, z) - f(x, \tilde{z})| \leq L|z - \tilde{z}| \quad \forall (x, z), (x, \tilde{z}) \in [x_0, X_M] \times \mathbb{R},$$

Explicit one-step methods

A general **explicit one-step method** can be written as

$$y_{n+1} = y_n + h\Phi(x_n, y_n; h, f), \quad n \in \{0, \dots, N-1\}, \quad y_0 = y(x_0),$$

where $\Phi = \Phi(\xi, \eta; h, f)$ is a continuous fct in the arguments $\xi, \eta; h$.
In this case,

$$\Psi(\xi, \eta; h, f) = \eta + h\Phi(\xi, \eta; h, f).$$

Example: for explicit Euler, we have $\Phi(\xi, \eta; h, f) = f(\xi, \eta)$ and $\Psi(\xi, \eta; h, f) = \eta + hf(\xi, \eta)$.

Rk: $\Phi(\xi, \eta; h, f)$ and $\Psi(\xi, \eta; h, f)$ can be explicitly computed.

From now on: we do not indicate the dependence of $\Phi(\xi, \eta; h, f)$ on f , and will write $\Phi(\xi, \eta; h)$ instead. E.g., for explicit Euler: $\Phi(\xi, \eta; h) = f(\xi, \eta)$.

4. General explicit one-step methods

Global error and consistency error

Recall: a general **explicit one-step method** can be written as

$$y_{n+1} = y_n + h\Phi(x_n, y_n; h), \quad n \in \{0, \dots, N-1\}, \quad y_0 = y(x_0),$$

where $\Phi(\cdot, \cdot; \cdot)$ is a continuous fct.

We define the **global error**, e_n , by

$$e_n := y(x_n) - y_n, \quad n \in \{0, \dots, N\}.$$

We define the **consistency error**, T_n , by

$$T_n = \frac{y(x_{n+1}) - y(x_n)}{h} - \Phi(x_n, y(x_n); h), \quad n \in \{0, \dots, N-1\}.$$

Rk: For an implicit one-step method $y_{n+1} = y_n + h\Phi(x_n, y_n, y_{n+1}; h)$, the consistency error is defined by

$$T_n = \frac{y(x_{n+1}) - y(x_n)}{h} - \Phi(x_n, y(x_n), y(x_{n+1}); h), \quad n \in \{0, \dots, N-1\}.$$

An error bound

Theorem (Error bound for general explicit one-step methods)

Consider the general one-step method

$$y_{n+1} = y_n + h\Phi(x_n, y_n; h), \quad n \in \{0, \dots, N-1\}, \quad y_0 = y(x_0),$$

where, in addition to being a continuous fct, Φ is assumed to satisfy a Lipschitz condition (w.r.t. its 2nd arg.); namely, $\exists L_\Phi, h_0 > 0$ s.t., for $h \in [0, h_0]$ and for the same region R as in Picard's Theorem,

$$|\Phi(x, z; h) - \Phi(x, \tilde{z}; h)| \leq L_\Phi |z - \tilde{z}|, \quad \forall (x, z), (x, \tilde{z}) \in R.$$

Assume that $|y_n - y_0| \leq Y_M \forall n \in \{0, \dots, N\}$. Then,

$$|e_n| \leq e^{L_\Phi(x_n - x_0)} |e_0| + \frac{e^{L_\Phi(x_n - x_0)} - 1}{L_\Phi} T, \quad n \in \{0, \dots, N\},$$

where $T := \max_{n \in \{0, \dots, N-1\}} |T_n|$.

Proof of the error bound

From the defn of the method and the defn of T_n , we have

$$y_{n+1} = y_n + h\Phi(x_n, y_n; h), \quad y(x_{n+1}) = y(x_n) + h\Phi(x_n, y(x_n); h) + hT_n.$$

Subtracting the first equality from the second, we find that

$$e_{n+1} = e_n + h[\Phi(x_n, y(x_n); h) - \Phi(x_n, y_n; h)] + hT_n.$$

Since $(x_n, y(x_n)), (x_n, y_n) \in R$, the Lipschitz condition implies

$$|e_{n+1}| \leq |e_n| + hL_\Phi |e_n| + h|T_n| \leq (1 + hL_\Phi)|e_n| + hT.$$

By induction, we then can show

$$|e_n| \leq (1 + hL_\Phi)^n |e_0| + \frac{(1 + hL_\Phi)^n - 1}{L_\Phi} T.$$

Finally, using $1 + x \leq e^x \quad \forall x \in \mathbb{R}$, and $nh = x_n - x_0$, we have

$$|e_n| \leq e^{nhL_\Phi} |e_0| + \frac{e^{nhL_\Phi} - 1}{L_\Phi} T = e^{L_\Phi(x_n - x_0)} |e_0| + \frac{e^{L_\Phi(x_n - x_0)} - 1}{L_\Phi} T.$$



Applying the thm to a specific IVP solved by expl. Euler

Consider the IVP

$$y'(x) = \arctan(y(x)) \quad \text{for } x \in (0, 1), \quad y(0) = 1.$$

Here, $f : \mathbb{R}^2 \rightarrow \mathbb{R}$, $f(x, z) := \arctan(z)$. (Exercise: Show $\exists!$ soln on $[0, 1]$.)
Use explicit Euler to approximate the soln $\implies \Phi(x, z; h) := f(x, z)$.

Compute a Lip. const. L_Φ : $|\partial_z f(x, z)| = \frac{1}{1+z^2} \leq 1 \quad \forall (x, z) \in \mathbb{R}^2$.
 $\implies \Phi$ satisfies a global Lip. cond. with $L_\Phi := 1$.

Rk: As Φ satisfies a global Lip.cond., we see from the proof of the general error bound that the assumption $|y_n - y_0| \leq Y_M \quad \forall n$ is not necessary.

The general error bound gives for our case (using $e_0 = 0$):

$$|e_n| \leq \frac{e^{L_\Phi(x_n - x_0)} - 1}{L_\Phi} T \leq (e^{x_n} - 1) \frac{M_2}{2} h, \quad M_2 := \max_{x \in [0, 1]} |y''(x)|,$$

where we have used that $T = \max_{n \in \{0, \dots, N-1\}} |T_n| \leq \frac{M_2}{2} h$ for expl. Euler.

Applying the thm to a specific IVP solved by expl. Euler

Consider the IVP

$$y'(x) = \arctan(y(x)) \quad \text{for } x \in (0, 1), \quad y(0) = 1.$$

Here, $f : \mathbb{R}^2 \rightarrow \mathbb{R}$, $f(x, z) := \arctan(z)$. Use explicit Euler to approximate the soln. We have shown

$$|e_n| \leq (e^{x_n} - 1) \frac{M_2}{2} h, \quad M_2 := \max_{x \in [0, 1]} |y''(x)|.$$

Let us bound M_2 : For any $x \in [0, 1]$, we have

$$|y''(x)| = \left| \frac{d}{dx} (\arctan(y(x))) \right| = \frac{|y'(x)|}{1 + [y(x)]^2} = \frac{|\arctan(y(x))|}{1 + [y(x)]^2} \leq \frac{\pi}{2}.$$

So, $M_2 \leq \frac{\pi}{2}$ and thus,

$$|e_n| \leq (e^{x_n} - 1) \frac{\pi}{4} h \leq \frac{\pi(e - 1)}{4} h \quad \forall n \in \{0, \dots, N\}.$$

$\implies$ For given $\text{TOL} > 0$, we have $|e_n| \leq \text{TOL} \quad \forall n$ when $h \leq \frac{4}{\pi(e - 1)} \text{TOL}$.

Consistency

Error bd for general explicit one-step method (assuming $e_0 = 0$):

$$|e_n| \leq \frac{e^{L_\Phi(x_n - x_0)} - 1}{L_\Phi} T, \quad n \in \{0, \dots, N\}.$$

$\implies$ Consistency error “decides” whether global error converges to 0.

Definition (Consistent method)

An explicit one-step scheme is **consistent** with the DE if the consistency error is such that $\forall \varepsilon > 0 \exists h_\varepsilon > 0$ s.t. $|T_n| < \varepsilon$ for all $h \in (0, h_\varepsilon)$ and any points $(x_n, y(x_n))$, $(x_{n+1}, y(x_{n+1}))$ on the graph of y .

Recall the defn

$$T_n = \frac{y(x_{n+1}) - y(x_n)}{h} - \Phi(x_n, y(x_n); h).$$

We see that for any $x \in [x_0, X_M]$, we have

$$\lim_{h \rightarrow 0, n \rightarrow \infty, x_n \rightarrow x \in [x_0, X_M]} T_n = y'(x) - \Phi(x, y(x); 0) = f(x, y(x)) - \Phi(x, y(x); 0)$$

$\implies$ the one-step method is consistent iff $\Phi(x, y; 0) \equiv f(x, y)$.

Convergence theorem for general explicit one-step methods

Theorem

Suppose the true soln of the IVP and its approximation lie in R when $h \leq h_0$. Suppose also that $\Phi(\cdot, \cdot; \cdot)$ is uniformly continuous on $R \times [0, h_0]$ and satisfies the consistency condition $\Phi(x, y; 0) \equiv f(x, y)$ and the Lipschitz condition

$$|\Phi(x, z; h) - \Phi(x, \tilde{z}; h)| \leq L_\Phi |z - \tilde{z}| \quad \forall (x, z, h), (x, \tilde{z}, h) \in R \times [0, h_0].$$

Then, if successive approximation sequences (y_n) , generated for $x_n = x_0 + nh$, $n \in \{1, \dots, N\}$, are obtained from the method with successively smaller values of h , each less than h_0 , we have convergence of the numerical solution to the solution of the IVP in the sense that

$$|y(x) - y_n| \longrightarrow 0 \quad \text{as} \quad h \rightarrow 0, n \rightarrow \infty, x_n \rightarrow x \in [x_0, X_M].$$

Order of accuracy of explicit one-step methods

We saw that for explicit Euler we have $|T_n| = \mathcal{O}(h)$, i.e., $\exists h_0, K > 0$ s.t.

$$|T_n| \leq Kh \quad \forall h \in (0, h_0].$$

However, there are other one-step methods (which we will discuss later) for which we can do better ($\mathcal{O}(h^2), \mathcal{O}(h^3), \dots$).

Definition (Order of accuracy)

The method is said to have **order of accuracy** p (or **order of consistency** p), if $p \in \mathbb{N}$ is the largest natural number s.t., for any sufficiently smooth solution curve $(x, y(x))$ in R of the IVP we have

$$|T_n| = \mathcal{O}(h^p),$$

i.e., there exist constants $h_0, K > 0$ such that $|T_n| \leq Kh^p$ for all $h \in (0, h_0]$, for any pair of points $(x_n, y(x_n)), (x_{n+1}, y(x_{n+1}))$ on the solution curve.

5 Explicit Runge–Kutta methods (optional)

Explicit Runge–Kutta methods

Motivation: explicit Euler is only first-order accurate (but cheap to implement because we only need one evaluation of f in each step).

Runge–Kutta (RK) methods: **higher accuracy by sacrificing the efficiency of explicit Euler through re-evaluating f :**

General **R -stage explicit RK family:** $y_{n+1} = y_n + h\Phi(x_n, y_n; h)$, where

$$\begin{aligned}\Phi(x, z; h) &= \sum_{r=1}^R c_r k_r(x, z; h), \\ k_1(x, z; h) &= f(x, z), \\ k_r(x, z; h) &= f\left(x + ha_r, z + h \sum_{s=1}^{r-1} b_{rs} k_s(x, z; h)\right), \quad r \in \{2, \dots, R\}.\end{aligned}$$

General Runge–Kutta methods

General version of a R -stage RK method:

$$y_{n+1} = y_n + h \sum_{r=1}^R c_r k_r,$$

$$k_r = f \left(x_n + h a_r, y_n + h \sum_{s=1}^R b_{rs} k_s \right) \quad \text{for } r \in \{1, \dots, R\}.$$

If the method is not a R -stage explicit RK method, then it is called a **R -stage implicit RK method**.

A RK method is usually displayed in the so-called **Butcher tableau**

$$\begin{array}{c|c} a & B \\ \hline & c^T \end{array}$$

where

$$a = (a_1, \dots, a_R)^T, \quad B = (b_{ij})_{1 \leq i, j \leq R}, \quad c = (c_1, \dots, c_R)^T.$$

For explicit RK methods, the matrix B is strictly lower-triangular.

1-stage explicit RK methods

Suppose that $R = 1$, i.e., $y_{n+1} = y_n + h\Phi(x_n, y_n; h)$, where

$$\begin{aligned}\Phi(x, z; h) &= c_1 k_1(x, z; h), \\ k_1(x, z; h) &= f(x, z).\end{aligned}$$

$$\implies y_{n+1} = y_n + hc_1 f(x_n, y_n).$$

The method is consistent iff $\Phi(x, y; 0) \equiv f(x, y)$, i.e., iff $c_1 = 1$.

The resulting one-stage explicit RK method is the explicit Euler method:

$$y_{n+1} = y_n + hf(x_n, y_n).$$

This is **first-order accurate**. Butcher tableau:

$$\begin{array}{c|c} 0 & 0 \\ \hline & 1 \end{array}$$

2-stage explicit RK methods

Suppose that $R = 2$, i.e., $y_{n+1} = y_n + h\Phi(x_n, y_n; h)$, where

$$\begin{aligned}\Phi(x, z; h) &= c_1 k_1(x, z; h) + c_2 k_2(x, z; h), \\ k_1(x, z; h) &= f(x, z), \\ k_2(x, z; h) &= f(x + a_2 h, z + b_{21} h k_1(x, z; h)).\end{aligned}$$

Q: Can we choose a_2, b_{21}, c_1, c_2 s.t. the method is second-order accurate (or even better)?

Consistency $\iff \Phi(x, y; 0) \equiv f(x, y) \iff c_1 f(x, y) + c_2 f(x, y) \equiv f(x, y)$,
i.e., **the method is consistent iff $c_1 + c_2 = 1$.**

Consistency error: $T_n = \frac{y(x_{n+1}) - y(x_n)}{h} - \Phi(x_n, y(x_n); h)$, i.e.,

$$T_n = \frac{y(x_{n+1}) - y(x_n)}{h} - c_1 f(x_n, y(x_n)) - c_2 f(x_n + a_2 h, y(x_n) + b_{21} h f(x_n, y(x_n))).$$

2-stage explicit RK methods: Expansion of T_n

We want to expand T_n in powers of h . Recall:

$$T_n = \frac{y(x_{n+1}) - y(x_n)}{h} - c_1 f(x_n, y(x_n)) - c_2 f(x_n + a_2 h, y(x_n) + b_{21} h f(x_n, y(x_n)))$$

Taylor expansion for the **first term**:

$$\frac{y(x_{n+1}) - y(x_n)}{h} = y'(x_n) + \frac{1}{2} h y''(x_n) + \frac{1}{6} h^2 y'''(x_n) + \mathcal{O}(h^3).$$

Taylor expansion for the **third term**:

$$\begin{aligned} & f(x_n + a_2 h, y(x_n) + b_{21} h f(x_n, y(x_n))) \\ &= \left[f + a_2 h f_x + b_{21} h f f_z + \frac{a_2^2}{2} h^2 f_{xx} + a_2 b_{21} h^2 f f_{xz} + \frac{b_{21}^2}{2} h^2 f^2 f_{zz} \right] (x_n, y(x_n)) \\ &+ \mathcal{O}(h^3). \end{aligned}$$

Noting that $y'(x_n) = f(x_n, y(x_n))$ and recalling $c_1 + c_2 = 1$, we find

$$\begin{aligned} T_n &= \frac{1}{2} h y''(x_n) + \frac{1}{6} h^2 y'''(x_n) - c_2 h [a_2 f_x + b_{21} f f_z](x_n, y(x_n)) \\ &\quad - c_2 h^2 \left[\frac{1}{2} a_2^2 f_{xx} + a_2 b_{21} f f_{xz} + \frac{1}{2} b_{21}^2 f^2 f_{zz} \right] (x_n, y(x_n)) + \mathcal{O}(h^3). \end{aligned}$$

We have obtained that

$$T_n = \frac{1}{2}h y''(x_n) + \frac{1}{6}h^2 y'''(x_n) - c_2 h [a_2 f_x + b_{21} f f_z](x_n, y(x_n)) \\ - c_2 h^2 \left[\frac{1}{2} a_2^2 f_{xx} + a_2 b_{21} f f_{xz} + \frac{1}{2} b_{21}^2 f^2 f_{zz} \right] (x_n, y(x_n)) + \mathcal{O}(h^3).$$

To continue, note that from $y'(x) = f(x, y(x))$, we find

$$y''(x) = f_x(x, y(x)) + y'(x) f_z(x, y(x)) = [f_x + f f_z](x, y(x)) = F_1(x, y(x)), \\ y'''(x) = [f_{xx} + f_x f_z + f f_{xz}](x, y(x)) + y'(x) [f_{xz} + f_z^2 + f f_{zz}](x, y(x)) \\ = [f_x f_z + f f_z^2 + f_{xx} + 2 f f_{xz} + f^2 f_{zz}](x, y(x)) = [f_z F_1 + F_2](x, y(x)),$$

where $F_1 := f_x + f f_z$ and $F_2 := f_{xx} + 2 f f_{xz} + f^2 f_{zz}$. We obtain

$$T_n = h \left[\frac{1}{2} F_1 - a_2 c_2 f_x - b_{21} c_2 f f_z \right] (x_n, y(x_n)) \\ + h^2 \left[\frac{1}{6} f_z F_1 + \frac{1}{6} F_2 - \frac{1}{2} a_2^2 c_2 f_{xx} - a_2 b_{21} c_2 f f_{xz} + \frac{1}{2} b_{21}^2 c_2 f^2 f_{zz} \right] (x_n, y(x_n)) \\ + \mathcal{O}(h^3).$$

2-stage explicit RK methods: Conditions for $T_n = \mathcal{O}(h^2)$

We have obtained that

$$\begin{aligned} T_n = & h \left[\frac{1}{2} F_1 - a_2 c_2 f_x - b_{21} c_2 f f_z \right] (x_n, y(x_n)) \\ & + h^2 \left[\frac{1}{6} f_z F_1 + \frac{1}{6} F_2 - \frac{1}{2} a_2^2 c_2 f_{xx} - a_2 b_{21} c_2 f f_{xz} + \frac{1}{2} b_{21}^2 c_2 f^2 f_{zz} \right] (x_n, y(x_n)) \\ & + \mathcal{O}(h^3), \end{aligned}$$

where $F_1 := f_x + f f_z$ and $F_2 := f_{xx} + 2 f f_{xz} + f^2 f_{zz}$.

Q: Can we get $T_n = \mathcal{O}(h^2)$ for any f ?

$\implies$ Yes! Choose the parameters s.t. $a_2 c_2 = \frac{1}{2}$, $b_{21} c_2 = \frac{1}{2}$, $c_1 + c_2 = 1$,
i.e.,

$$b_{21} = a_2, \quad c_2 = \frac{1}{2a_2}, \quad c_1 = 1 - \frac{1}{2a_2}.$$

This still leaves one free parameter, a_2 , but **no choice of the parameters** will yield $T_n = \mathcal{O}(h^3)$ for any f .

Examples of second-order explicit RK methods

Recall general 2-stage explicit RK method:

$$y_{n+1} = y_n + h [c_1 f(x_n, y_n) + c_2 f(x_n + a_2 h, y_n + b_{21} h f(x_n, y_n))].$$

Recall: if $b_{21} = a_2$, $c_2 = \frac{1}{2a_2}$, $c_1 = 1 - \frac{1}{2a_2}$, then second-order accurate.

■ **The modified Euler method:** $a_2 := \frac{1}{2}$, $b_{21} := \frac{1}{2}$, $c_1 := 0$, $c_2 := 1$.

$$y_{n+1} = y_n + h f \left(x_n + \frac{h}{2}, y_n + \frac{h}{2} f(x_n, y_n) \right).$$

0	0 0
$\frac{1}{2}$	$\frac{1}{2}$ 0
	0 1

■ **The improved Euler method:** $a_2 := 1$, $b_{21} := 1$, $c_1 := \frac{1}{2}$, $c_2 := \frac{1}{2}$.

$$y_{n+1} = y_n + \frac{h}{2} [f(x_n, y_n) + f(x_n + h, y_n + h f(x_n, y_n))].$$

0	0 0
1	1 0
	$\frac{1}{2}$ $\frac{1}{2}$

3-stage explicit RK methods: the autonomous case

Suppose that $R = 3$ and $f(x, z) = \tilde{f}(z)$ is independent of x (the ODE $y'(x) = \tilde{f}(y(x))$ is called **autonomous**). Then,

$$y_{n+1} = y_n + h\Phi(x_n, y_n; h),$$

where $\Phi(x, z; h) = c_1\tilde{k}_1(z; h) + c_2\tilde{k}_2(z; h) + c_3\tilde{k}_3(z; h)$ and

$$\tilde{k}_1(z; h) = \tilde{f}(z),$$

$$\tilde{k}_2(z; h) = \tilde{f}(z + hb_{21}\tilde{k}_1(z; h)),$$

$$\tilde{k}_3(z; h) = \tilde{f}(z + hb_{31}\tilde{k}_1(z; h) + hb_{32}\tilde{k}_2(z; h)).$$

Q: Can we choose the parameters s.t. the method is third-order accurate (or even better)?

Consistency $\iff \Phi(x, y; 0) \equiv \tilde{f}(y) \iff c_1\tilde{f}(y) + c_2\tilde{f}(y) + c_3\tilde{f}(y) \equiv \tilde{f}(y)$
 $\iff c_1 + c_2 + c_3 = 1$.

Consistency error: $T_n = \frac{y(x_{n+1}) - y(x_n)}{h} - \Phi(x_n, y(x_n); h)$, i.e.,

$$T_n = \frac{y(x_{n+1}) - y(x_n)}{h} - c_1\tilde{f}(y(x_n)) - c_2\tilde{k}_2(y(x_n); h) - c_3\tilde{k}_3(y(x_n); h).$$

3-stage exp. RK methods, autonomous case: expanding T_n

We want to expand T_n in powers of h . Recall:

$$T_n = \frac{y(x_{n+1}) - y(x_n)}{h} - c_1 \tilde{f}(y(x_n)) - c_2 \tilde{k}_2(y(x_n); h) - c_3 \tilde{k}_3(y(x_n); h)$$

Taylor expansion for $\tilde{k}_2(y(x_n); h)$:

$$\tilde{k}_2(y(x_n); h) := \tilde{f}(y(x_n) + hb_{21}\tilde{f}(y(x_n))) = \left[\tilde{f} + hb_{21}\tilde{f}\tilde{f}' + h^2 \frac{b_{21}^2}{2} \tilde{f}^2 \tilde{f}'' \right] (y(x_n)) + \mathcal{O}(h^3).$$

Taylor expansion for $\tilde{k}_3(y(x_n); h)$:

$$\begin{aligned} k_3(y(x_n); h) &:= \tilde{f}(y(x_n) + hb_{31}\tilde{f}(y(x_n)) + hb_{32}\tilde{k}_2(y(x_n); h)) \\ &= \left[\tilde{f} + h \left(b_{31}\tilde{f}\tilde{f}' + b_{32}(\tilde{f} + hb_{21}\tilde{f}\tilde{f}' + \mathcal{O}(h^2))\tilde{f}' \right) + h^2 \left(\frac{1}{2}(b_{31}\tilde{f} + b_{32}(\tilde{f} + \mathcal{O}(h)))^2 \tilde{f}'' \right) \right] (y(x_n)) \\ &\quad + \mathcal{O}(h^3) \\ &= \left[\tilde{f} + h \left((b_{31} + b_{32})\tilde{f}\tilde{f}' \right) + h^2 \left(b_{21}b_{32}\tilde{f}\tilde{f}'^2 + \frac{1}{2}(b_{31} + b_{32})^2 \tilde{f}^2 \tilde{f}'' \right) \right] (y(x_n)) + \mathcal{O}(h^3). \end{aligned}$$

We find that

$$\begin{aligned}\Phi(x_n, y(x_n); h) &= c_1 \tilde{f}(y(x_n)) + c_2 \tilde{k}_2(y(x_n); h) + c_3 \tilde{k}_3(y(x_n); h) \\ &= (c_1 + c_2 + c_3) \tilde{f}(y(x_n)) + h \left[(b_{21} c_2 + (b_{31} + b_{32}) c_3) \tilde{f} \tilde{f}' \right] (y(x_n)) \\ &\quad + h^2 \left[b_{21} b_{32} c_3 \tilde{f} \tilde{f}'^2 + \frac{b_{21}^2 c_2 + (b_{31} + b_{32})^2 c_3}{2} \tilde{f}^2 \tilde{f}'' \right] (y(x_n)) + \mathcal{O}(h^3)\end{aligned}$$

and we also have that

$$\begin{aligned}\frac{y(x_{n+1}) - y(x_n)}{h} &= y'(x_n) + \frac{1}{2} h y''(x_n) + \frac{1}{6} h^2 y'''(x_n) + \mathcal{O}(h^3) \\ &= 1 \cdot \tilde{f}(y(x_n)) + h \left[\frac{1}{2} \tilde{f} \tilde{f}' \right] (y(x_n)) + h^2 \left[\frac{1}{6} \tilde{f} \tilde{f}'^2 + \frac{1}{6} \tilde{f}^2 \tilde{f}'' \right] (y(x_n)) + \mathcal{O}(h^3).\end{aligned}$$

Noting that $T_n = \frac{y(x_{n+1}) - y(x_n)}{h} - \Phi(x_n, y(x_n); h)$, we conclude that we can achieve $T_n = \mathcal{O}(h^3)$ if we choose the parameters s.t.

$$\begin{aligned}c_1 + c_2 + c_3 &= 1, & b_{21} c_2 + (b_{31} + b_{32}) c_3 &= \frac{1}{2}, \\ b_{21}^2 c_2 + (b_{31} + b_{32})^2 c_3 &= \frac{1}{3}, & b_{21} b_{32} c_3 &= \frac{1}{6}.\end{aligned}$$

Solving for the six unknowns, we obtain a two-parameter family of third-order accurate 3-stage explicit RK methods.

Examples of third-order explicit RK methods

Recall third-order accuracy conditions (for the autonomous case):

$$\begin{aligned}c_1 + c_2 + c_3 &= 1, & b_{21}c_2 + (b_{31} + b_{32})c_3 &= \frac{1}{2}, \\b_{21}^2c_2 + (b_{31} + b_{32})^2c_3 &= \frac{1}{3}, & b_{21}b_{32}c_3 &= \frac{1}{6},\end{aligned}$$

- **Heun's method:** $c_1 = \frac{1}{4}$, $c_2 = 0$, $c_3 = \frac{3}{4}$, $b_{21} = \frac{1}{3}$, $b_{31} = 0$, $b_{32} = \frac{2}{3}$.

$$y_{n+1} = y_n + h \left(\frac{1}{4} \tilde{k}_1 + \frac{3}{4} \tilde{k}_3 \right),$$

$$\tilde{k}_1 = \tilde{f}(y_n), \quad \tilde{k}_2 = \tilde{f}\left(y_n + \frac{1}{3}h\tilde{k}_1\right), \quad \tilde{k}_3 = \tilde{f}\left(y_n + \frac{2}{3}h\tilde{k}_2\right).$$

- **Standard third-order explicit RK method:** $c_1 = \frac{1}{6}$, $c_2 = \frac{2}{3}$, $c_3 = \frac{1}{6}$, $b_{21} = \frac{1}{2}$, $b_{31} = -1$, $b_{32} = 2$.

R-stage explicit RK methods with $R \geq 4$

For $R = 4$, an analogous argument leads to a two-parameter family of four-stage RK methods of order four. Popular example:

$$y_{n+1} = y_n + \frac{1}{6}h(k_1 + 2k_2 + 2k_3 + k_4),$$

where

$$\begin{aligned}k_1 &= f(x_n, y_n), & k_2 &= f\left(x_n + \frac{1}{2}h, y_n + \frac{1}{2}hk_1\right), \\k_3 &= f\left(x_n + \frac{1}{2}h, y_n + \frac{1}{2}hk_2\right), & k_4 &= f(x_n + h, y_n + hk_3).\end{aligned}$$

We have constructed R -stage explicit RK methods of order of accuracy $\mathcal{O}(h^R)$, $R = 1, 2, 3, 4$.

Q: Is there an R -stage method of order R for $R \geq 5$?

$\Rightarrow$ **No.** For $R = 5, 6, 7, 8, 9$, the highest order that can be attained by a R -stage RKm is 4, 5, 6, 6, 7. For $R \geq 10$, highest order is $\leq R - 2$.

6 Absolute stability of explicit Runge–Kutta methods (optional)

The model problem

We consider the **model problem**¹

$$y'(x) = \lambda y(x), \quad y(0) = y_0,$$

with $\lambda < 0$ and $y_0 \neq 0$. The true solution to this IVP is

$$y(x) = y_0 e^{\lambda x}.$$

Observe that $y(x) \rightarrow 0$ as $x \rightarrow \infty$.

Q: Under what conditions on the step size h does a RK method reproduce this behavior?

$\implies$ the answer gives information on how to choose h in the approximation of an IVP by an explicit RK method over $[x_0, X_M]$ with $X_M \gg x_0$.

We answer this question for R -stage explicit RK methods of order of accuracy R , for $R = 1, 2, 3, 4$.

¹also called the Dahlquist's test equation

Absolute stability of 1st-order accurate 1-stage explicit RK

The only first-order accurate 1-stage explicit RK method is explicit Euler:

$$y_{n+1} = y_n + hf(x_n, y_n).$$

We apply this to the model problem

$$y'(x) = \lambda y(x), \quad y(0) = y_0.$$

Here, $f : \mathbb{R}^2 \rightarrow \mathbb{R}$, $f(x, z) := \lambda z$. We find

$$y_{n+1} = y_n + hf(x_n, y_n) = y_n + \lambda h y_n = (1 + \bar{h})y_n,$$

where $\bar{h} := \lambda h$. Thus, for any $n \in \mathbb{N}_0$ we have

$$y_n = (1 + \bar{h})^n y_0.$$

$\implies (y_n)_{n \in \mathbb{N}_0}$ will converge to 0 iff $|1 + \bar{h}| < 1$, i.e., iff $\bar{h} \in (-2, 0)$.

For h chosen s.t. $\bar{h} \in (-2, 0)$, explicit Euler is said to be **absolutely stable**, and $(-2, 0)$ is called **interval of absolute stability** of the method.

Absolute stability of 2nd-order accurate 2-stage explicit RK

Consider a second-order accurate 2-stage explicit RK method:

$$y_{n+1} = y_n + h(c_1 k_1 + c_2 k_2), \quad k_1 = f(x_n, y_n), \quad k_2 = f(x_n + a_2 h, y_n + b_{21} h k_1)$$

with parameters satisfying $c_1 + c_2 = 1$ and $a_2 c_2 = b_{21} c_2 = \frac{1}{2}$ (to have second-order accuracy). Apply this to model problem ($f(x, z) = \lambda z$):

$$\begin{aligned} y_{n+1} &= y_n + h(c_1 \lambda y_n + c_2 \lambda (y_n + b_{21} h \lambda y_n)) \\ &= (1 + (c_1 + c_2) \lambda h + b_{21} c_2 \lambda^2 h^2) y_n = \left(1 + \bar{h} + \frac{1}{2} \bar{h}^2\right) y_n. \end{aligned}$$

(Recall $\bar{h} := \lambda h$.) Thus, for any $n \in \mathbb{N}_0$ we have

$$y_n = \left(1 + \bar{h} + \frac{1}{2} \bar{h}^2\right)^n y_0.$$

$\implies$ Method absolutely stable ($y_n \rightarrow 0$ as $n \rightarrow \infty$) iff $\left|1 + \bar{h} + \frac{1}{2} \bar{h}^2\right| < 1$

iff $\frac{1}{2}(\bar{h} + 1)^2 + \frac{1}{2} < 1$ iff $(\bar{h} + 1)^2 < 1$ iff $\bar{h} + 1 \in (-1, 1)$ iff $\bar{h} \in (-2, 0)$.

Absolute stability of 3rd-order accurate 3-stage explicit RK

For third-order accurate 3-stage explicit RK methods, an analogous argument yields

$$y_{n+1} = \left(1 + \bar{h} + \frac{1}{2}\bar{h}^2 + \frac{1}{6}\bar{h}^3\right) y_n$$

and hence, for any $n \in \mathbb{N}_0$,

$$y_n = \left(1 + \bar{h} + \frac{1}{2}\bar{h}^2 + \frac{1}{6}\bar{h}^3\right)^n y_0.$$

$\implies$ Method absolutely stable ($y_n \rightarrow 0$ as $n \rightarrow \infty$) iff

$$\left|1 + \bar{h} + \frac{1}{2}\bar{h}^2 + \frac{1}{6}\bar{h}^3\right| < 1,$$

which yields the interval of absolute stability: $\bar{h} \in (-a, 0)$ with $a \approx 2.51$.

Absolute stability of 4th-order accurate 4-stage explicit RK

For fourth-order accurate 4-stage explicit RK methods, an analogous argument yields

$$y_{n+1} = \left(1 + \bar{h} + \frac{1}{2}\bar{h}^2 + \frac{1}{6}\bar{h}^3 + \frac{1}{24}\bar{h}^4\right) y_n,$$

and the interval of absolute stability is $\bar{h} \in (-a, 0)$ with $a \approx 2.78$.

Rk: For $R \geq 5$, applying the explicit RK method to model problem gives

$$y_{n+1} = A_R(\bar{h})y_n,$$

however, unlike the cases $R = 1, 2, 3, 4$, in addition to $\bar{h}$ the expression $A_R(\bar{h})$ also depends on the coefficients of the explicit RK method.

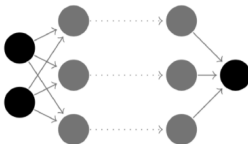
End of "Section 2: One-step methods".

4.3 From ODE to DL

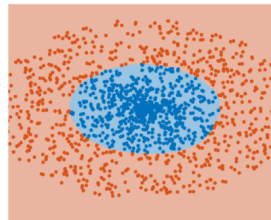
Example: Supervised Classification with a DNN



training data



DNN sketch

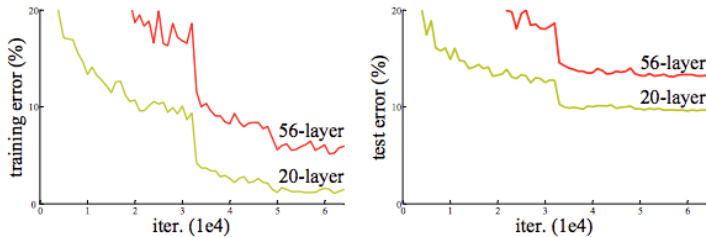


classification result

1. ResNet: Residual Neural Networks (He et al. 2016)

Motivation

Evidence shows that the best ImageNet models using convolutional and fully-connected layers typically contain between 16 and 30 layers.



Hard to train deep networks!

During each iteration of training a neural network, all weights receive an update proportional to the partial derivative of the loss function w.r.t. the current weight. If the gradient is very small then the weights will not be change effectively and it may completely stop the neural network from further training. The phenomenon is called *vanishing gradients*.

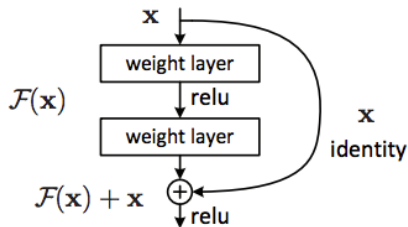
Difficulties in deep learning

Deep learning is a type of machine learning that uses neural networks with many hidden layers, which has been successful in supervised learning, particularly when the relation between the data and the labels is highly nonlinear.

- using deeper network architectures increases the capacity of the network but also the dimensionality and thus the **computational complexity**
- **more labeled data** is required to calibrate very deep networks reliably
- The associated optimization problem is **non-convex** and **high-dimensional**, and thus extremely challenging to solve; Moreover, the performance of DNN is highly dependent on the choice of the optimization algorithm and sample size.
- **instability of the underlying forward model**, possibly leading to **vanishing or exploding gradients**, and **bad generalization**

Residual learning: a building block

The problem of training very deep networks has been alleviated with the introduction of a new neural network layer — The Residual Block.



Skip Connection

the most important modification to understand is the 'Skip Connection', identity mapping.

ResNet: Residual Neural Networks (He et al. 2016)

Training data: $\{(y^{(1)}, c^{(1)}), (y^{(2)}, c^{(2)}), \dots\} \subset \mathbb{R}^2 \times \{0, 1\}$

Forward propagation of input y through simple ResNet

$$\begin{aligned} u_0 &= K_{\text{in}} y \\ u_1 &= u_0 + h\sigma(K_0 u_0 + b_0) \\ &\vdots \\ u_N &= u_{N-1} + h\sigma(K_{N-1} u_{N-1} + b_{N-1}) \\ z &= s(K_{\text{out}} u_N + b_{\text{out}}), \end{aligned}$$

with $h > 0$, and $\theta_i^{\text{Res}} := (K_i, b_i)$. Let $F(y, \theta) := z$, weight $\theta := (\theta_0^{\text{Res}}, \dots, \theta_{N-1}^{\text{Res}}, K_{\text{out}}, b_{\text{out}})$.

Train weights by solving (Bottou et al. 2018)

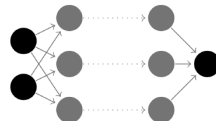
$$\min_{\theta} \mathbb{E}[\ell(F(y, \theta), c)] + \frac{\alpha}{2} \|\theta\|_2^2,$$

with cross entropy loss $\ell(z, c) := -c \log(z) - (1 - c) \log(1 - z)$.

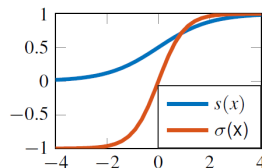
training data:



DNN sketch:



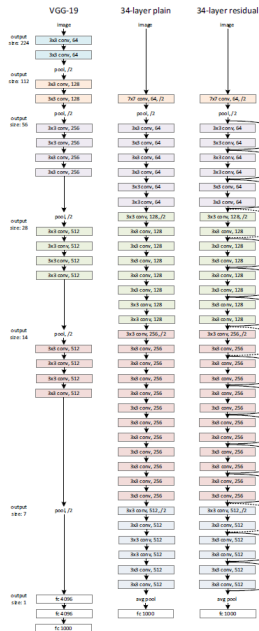
activation σ and sigmoid s :



Let N be the number of hidden layers, and V_N be the spaces of functions that can be expressed by this ResNet, then

$$V_1 \subset V_2 \subset \cdots \subset V_{N-1} \subset V_N \subset$$

- Nested ansatz space
- The **identity mapping** can be expressed simply
- It is hypothesized that it is **easier to optimize the residual mapping** than to optimize the original, unreferenced mapping



ResNet: Discussion

In ResNet, u_N is forward Euler approximation of $u(T)$,

$$\partial_t u(t) = f(u(t), \theta^{\text{ODE}}(t)), t \in (0, T], u(0) = u_0;$$

see (E 2017; Haber and Ruthotto 2017) ($\approx$ same time).

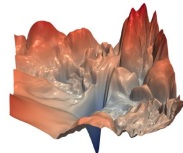
Advantages over other Architectures

- 1 ResNets often improve with depth
- 2 state-of-the-art results for many tasks
- 3 easy to train and easy to add depth

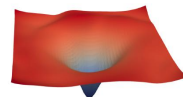
Remarks

- 1 $f(u(t), \theta^{\text{ODE}}(t)) = \sigma(K(t)u + b(t))$ gives ResNet on previous slide
- 2 in practice: more complicated layer f , concatenate ResNets to change width or image resolution
- 3 similar continuous networks, extensions to PDEs, and implicit time integrators in (Rico-Martínez et al. 1992; González-García et al. 1998).

impact on loss function
(Li et al. 2018)



56-layer network (no ResNet)



56-layer ResNet

Stable Architectures for DNNs (Haber and Ruthotto 2017)

When is forward propagation stable?

That is, when $\exists M > 0$ such that

$$\|F(y + \epsilon, \theta) - F(y, \theta)\| \leq M\|\epsilon\|, \quad \epsilon \text{ is the input perturbation}$$

Motivation: **well-posed training problem**, **adversarial attacks**², **efficient optimization**, etc.

Let the forward propagation be

$$Y_{j+1} = Y_j + h\sigma(Y_j K_j + b_j) \quad j = 0, \dots, N-1. \quad (10)$$

Here, $Y_0 = [y^{(1)}, y^{(2)}, \dots, y^{(s)}]^T$ is the stacked training feature, $K_j \in \mathbb{R}^{n \times n}$ and $b_j \in \mathbb{R}$ for $j = 0, \dots, N$.

²an attack where the goal is to cause an AI system to make a mistake or misclassification, often through subtle manipulations of the input data

The forward propagation (10) can be seen as an explicit Euler discretization of the first order explicit ODE,

$$\dot{u}(t) = f(u(t), \theta^{\text{ODE}}(t)) = \sigma(K(t)u + b(t)).$$

Then the forward propagation (10) is stable, if

$$\max_{i=1,2,\dots,n} |1 + h\lambda_i(J_j)| \leq 1, \quad \forall j = 0, 1, \dots, N-1. \quad (11)$$

Main Findings and Contributions

- 1 $\partial_t u(t) = \sigma(K(t)u(t) + b(t))$ not stable for all $K(\cdot)$, $b(\cdot)$
- 2 alternative f : antisymmetric K , Hamiltonian-inspired networks
- 3 symplectic integrators to obtain stable architecture ($\neq$ ResNet)
- 4 stable DNNs perform competitively (on simple tasks)

Improvements and Related Works

- 1 more expressive architectures (Chang et al. 2018), multilevel training (Chang et al. 2017)
- 2 improved stability results (Ruthotto and Haber 2020; Celledoni et al. 2020)
- 3 analysis: convergence (Thorpe and Gennip 2018), opt. conditions (Benning et al. 2019)
- 4 multi-step and other time integrators (Lu et al. 2017)
- 5 discrete weights (Li and Hao 2018), maximum principles (Li et al. 2017)

2. Neural ODEs: Neural Ordinary Differential Equations (Chen et al. 2018)

What is Neural ODEs?

Instead of specifying a discrete sequence of hidden layers as in Residual net, **Neural ODEs parameterize the derivative of the hidden state using a neural network.**

$$\frac{d}{dt}u(t) = f(u(t), \theta^{\text{ODE}}(t)), t \in (0, T], u(0) = u_0;$$

Here, f is a neural network!

Neural ODEs VS Residual networks

Neural ODEs

$$\frac{d\mathbf{x}(t)}{dt} = f(\mathbf{x}(t), t, \theta)$$

↓ Euler discretization

$$\mathbf{x}_{n+1} = \mathbf{x}_n + hf(\mathbf{x}_n, t_n, \theta)$$

Forward propagation:

$$\mathbf{x}(t_1) = \text{ODESolve}(f(\mathbf{x}(t), t, \theta), \mathbf{x}(t_0), t_0, t_1)$$

$$\ell(\mathbf{x}(t_1)) \rightarrow \frac{\partial \ell}{\partial \theta}, \text{ How to compute this?}$$

Update θ to reduce ℓ

Residual networks

$$\mathbf{x}_{l+1} = \text{ResBlock}(\mathbf{x}_l, \theta)$$

$$\mathbf{x}_{l+1} = \mathbf{x}_l + g(\mathbf{x}_l, \theta)$$

Skip connection

$$\mathbf{y}_{\text{pred}} = \text{ResNet}(\mathbf{x})$$

Stacked ResBlocks

$$\ell(\mathbf{y}_{\text{pred}}) \rightarrow \frac{\partial \ell}{\partial \theta}$$

Update θ to reduce ℓ

ODEs

$$\frac{d\mathbf{x}(t)}{dt} = f(\mathbf{x}(t), t, \theta)$$

↓ Euler discretization

$$\mathbf{x}_{n+1} = \mathbf{x}_n + hf(\mathbf{x}_n, t_n, \theta)$$

Forward propagation:

$$\mathbf{x}(t_1) = \text{ODESolve}(f(\mathbf{x}(t), t, \theta), \mathbf{x}(t_0), t_0, t_1)$$

$$\ell(\mathbf{x}(t_1)) \rightarrow \frac{\partial \ell}{\partial \theta}, \text{ How to compute this?}$$

Update θ to reduce ℓ

- Back-propagate through the ODE solver to calculate $\frac{\partial \ell}{\partial \theta}$!
- disadvantage: High memory cost- need to save all activations of all iterations of ODESolve
- we can save computation by the adjoint method.

Adjoint method (Pontryagin et al., 1962)

Question

$$\ell(\text{ODESolve}(f(\mathbf{x}(t), t, \theta), \mathbf{x}(t_0), t_0, t_1)) \rightarrow \frac{\partial \ell}{\partial \theta}$$

Adjoint method

Let $\mathbf{a}(t) := \frac{\partial \ell}{\partial \mathbf{x}(t)}$ and $a_\theta(t) = \frac{\partial \ell}{\partial \theta(t)}$ be the adjoint states, then they satisfy the following ODE,

$$\begin{aligned}\frac{d\mathbf{a}}{dt} &= -\mathbf{a}(t)^\top \frac{\partial f(\mathbf{x}(t), t, \theta)}{\partial \mathbf{x}(t)} \\ \frac{da_\theta}{dt} &= -\mathbf{a}(t)^\top \frac{\partial f(\mathbf{x}(t), t, \theta)}{\partial \theta(t)}.\end{aligned}$$

Forward propagation:

- $\mathbf{a}(t_1)$ can be calculated directly,

$$\mathbf{x}(t_1) = \text{ODESolve}(f(\mathbf{x}(t), t, \theta), \mathbf{x}(t_0), t_0, t_1)$$

$$\Rightarrow \mathbf{a}(t_1) := \frac{\partial \ell}{\partial \mathbf{x}(t_1)} \text{ is known}$$

- $\mathbf{a}_\theta(t_1) = \mathbf{0}$ since there is no weights in the output layer.

Consequently,

$$\frac{\partial \ell}{\partial \theta} = \int_{t_1}^{t_0} \mathbf{a}(t)^\top \frac{\partial f(\mathbf{x}(t), t, \theta)}{\partial \theta} dt$$

How to derive the adjoint method?

1/2.

Assume $\frac{\partial \theta(t)}{\partial t} = 0$, i.e. θ is time-invariant. By chain rule,

$$\begin{aligned} a(t) &= \frac{\partial \ell}{\partial x(t+\epsilon)} \frac{\partial x(t+\epsilon)}{\partial x(t)} \\ &= a(t+\epsilon) \frac{\partial}{\partial x(t)} \left(x(t) + \int_t^{t+\epsilon} f(x(s), s, \theta) ds \right) \\ &= a(t+\epsilon) + a(t+\epsilon) \frac{\partial}{\partial x(t)} \int_t^{t+\epsilon} f(x(s), s, \theta) ds \end{aligned}$$



How to derive the adjoint method?

2/2.

Then we have

$$\begin{aligned}\frac{da(t)}{dt} &= \lim_{\epsilon \rightarrow 0^+} \frac{a(t + \epsilon) - a(t)}{\epsilon} \\ &= \lim_{\epsilon \rightarrow 0^+} -\epsilon^{-1} a(t + \epsilon) \frac{\partial}{\partial x(t)} \int_t^{t+\epsilon} f(x(s), s, \theta(s)) ds \\ &= -a(t) \frac{\partial f(x(t), t, \theta(t))}{\partial x(t)}.\end{aligned}$$



Exercise

Derive the ODE satisfied by $a_\theta(t)$.

$$\frac{\partial \ell}{\partial \theta} = \int_{t_1}^{t_0} \mathbf{a}(t)^\top \frac{\partial f(\mathbf{x}(t), t, \theta)}{\partial \theta} dt$$

However, we do not know $\mathbf{a}(t)$ or $\frac{\partial f(\mathbf{x}(t), t, \theta)}{\partial \theta}$!

$$\ell(\text{ODESolve}(f(\mathbf{x}(t), t, \theta), \mathbf{x}(t_0), t_0, t_1)) \rightarrow \frac{\partial \ell}{\partial \theta}$$

$$\text{adjoint } \mathbf{a}(t) = \frac{\partial \ell}{\partial \mathbf{x}}; \frac{d\mathbf{a}}{dt} = -\mathbf{a}(t)^\top \frac{\partial f(\mathbf{x}(t), t, \theta)}{\partial \mathbf{x}}$$

Forward propagation:

$$\mathbf{x}(t_1) = \text{ODESolve}(f(\mathbf{x}(t), t, \theta), \mathbf{x}(t_0), t_0, t_1) \Rightarrow \mathbf{a}(t_1) = \frac{\partial \ell}{\partial \mathbf{x}(t_1)}$$

Back-propagation:

$$\blacksquare \mathbf{x}(t_0) = \text{ODESolve}(f(\mathbf{x}(t), t, \theta), \mathbf{x}(t_1), t_1, t_0)^3$$

$$\blacksquare \mathbf{a}(t_0) = \frac{\partial \ell}{\partial \mathbf{x}(t_0)} = \text{ODESolve}\left(-\mathbf{a}(t)^\top \frac{\partial f(\mathbf{x}(t), t, \theta)}{\partial \mathbf{x}}, \mathbf{a}(t_1), t_1, t_0\right)$$

Consequently,

$$\begin{aligned} \frac{\partial \ell}{\partial \theta} &= \int_{t_1}^{t_0} \mathbf{a}(t)^\top \frac{\partial f(\mathbf{x}(t), t, \theta)}{\partial \theta} dt \\ &= \text{ODESolve}\left(-\mathbf{a}(t)^\top \frac{\partial f(\mathbf{x}(t), t, \theta)}{\partial \theta}, \mathbf{0}, t_1, t_0\right) \end{aligned}$$

³Hence, no need to store intermediate quantities, and one can calculate the corresponding value as needed.

Reverse-mode derivative of an ODE initial value problem

Forward propagation

- $\mathbf{x}(t_1) = \text{ODESolve}(f(\mathbf{x}(t), t, \theta), \mathbf{x}(t_0), t_0, t_1)$
- Compute $\ell(\mathbf{x}(t_1))$
- $\mathbf{a}(t_1) = \frac{\partial \ell}{\partial \mathbf{x}(t_1)}$

Back propagation

$$\begin{bmatrix} \frac{\partial \ell}{\partial \mathbf{x}(t_0)} \\ \frac{\partial \ell}{\partial \theta} \end{bmatrix} = \text{ODESolve} \left(\begin{bmatrix} f(\mathbf{x}(t), t, \theta) \\ -\mathbf{a}(t)^\top \frac{\partial f(\mathbf{x}(t), t, \theta)}{\partial \mathbf{x}} \\ -\mathbf{a}(t)^\top \frac{\partial f(\mathbf{x}(t), t, \theta)}{\partial \theta} \end{bmatrix}, \begin{bmatrix} \frac{\partial \ell}{\partial \mathbf{x}(t_1)} \\ \mathbf{0} \end{bmatrix}, t_1, t_0 \right)$$

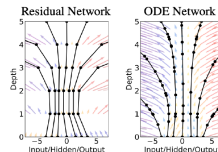
- The vector-Jacobian products $a(t)^T \frac{\partial f(x(t), t, \theta)}{\partial x(t)}$ and $a(t)^T \frac{\partial f(x(t), t, \theta)}{\partial \theta(t)}$ can be efficiently evaluated by automatic differentiation, at a time cost similar to that of evaluating $f(x(t), t, \theta)$.
- All integrals for solving $x(t)$, $a(t)$ and $a_\theta(t)$ can be computed in a single call to an ODE solver, which concatenates the original state, the adjoint states into a single vector.

Main Novelties and Contributions

- 1 apply adaptive time integrator to continuous ResNet
- 2 compute gradients of loss function using adjoint equation
- 3 save memory by re-computing $u(t)$ backward in time with p
- 4 popularized continuous models in ML community

Improvements and Related Works

- 1 example for numerical instability of item 3 above (Gholami et al. 2019) and alternative using checkpointing
- 2 invertible ResNet (Behrmann et al. 2019), generative modeling (Grathwohl et al. 2018; Chen et al. 2019)
- 3 augmented, more expressive models (Dupont et al. 2019)



from (Chen et al. 2018)

Artificial Intelligence / Machine Learning

A radical new neural network design could overcome big challenges in AI

Researchers borrowed equations from calculus to redesign the core machinery of deep learning so it can model continuous processes like changes in health.

by Karen Hao

December 12, 2018

MIT Tech Review, 2018

3. Neural ODEs: Applications

This section is based on the following literatures,

- Chen, Rubanova, Bettencourt, & Duvenaud (2018). *Neural ordinary differential equations*. *Advances in neural information processing systems*, 31.
- Msurtsukov. (n.d.-a). GitHub - msurtsukov/neural-ode: *Jupyter notebook with Pytorch implementation of Neural Ordinary Differential Equations*. GitHub.
<https://github.com/msurtsukov/neural-ode>.

1. Replace ResNet with Neural ODE for MNIST classifier

■ Performance

Table 1: Performance on MNIST. [†]From LeCun et al. (1998).

	Test Error	# Params	Memory	Time
1-Layer MLP [†]	1.60%	0.24 M	-	-
ResNet	0.41%	0.60 M	$\mathcal{O}(L)$	$\mathcal{O}(L)$
RK-Net	0.47%	0.22 M	$\mathcal{O}(\tilde{L})$	$\mathcal{O}(\tilde{L})$
ODE-Net	0.42%	0.22 M	$\mathcal{O}(1)$	$\mathcal{O}(\tilde{L})$

- ODE-Net is achieved by replacing the standard residual blocks of a ResNet by an ODESolve module.
- RK-Net: using the same ODE-Net architecture but gradients are backpropagated through a Runge-Kutta integrator.
- ODE-Net achieves same performance as ResNet with fewer parameters and less memory
- L : # layers in ResNet
- $\tilde{L}$: # function evaluations for ODE solver in a single forward pass $\Rightarrow$ implicit # layers in ODE-Net

Characteristics of Neural ODE in Supervised Learning

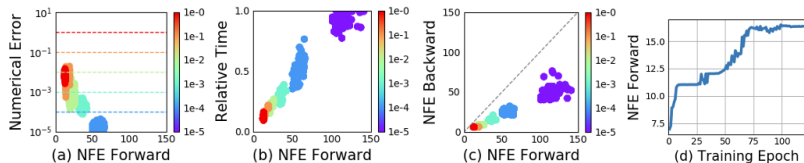


Figure 3: Statistics of a trained ODE-Net. (NFE = number of function evaluations.)

- **Explicit Error Control:** From (a)(b), changing the tolerable numerical error varies the number of steps per forward pass and hence time required
- **Computationally efficient:** From (c), # evaluations in the backward pass is roughly half of the forward pass $\Rightarrow$ **adjoint sensitivity method is more efficient than direct backward propagation**
- **Adaptive evaluation strategy:** From (d), # function evaluations increases during training $\Rightarrow$ adapting to increasing complexity of the model

2. Continuous Normalizing flows

■ Normalizing flow (NF)

$$\mathbf{z}_{t+1} = \mathbf{z}_t + f(\mathbf{z}_t) \Rightarrow \log p(\mathbf{z}_{t+1}) = \log p(\mathbf{z}_t) - \log \left| \det \left(I + \frac{\partial f}{\partial \mathbf{z}_t} \right) \right|$$

f should be bijective and determinant is costly

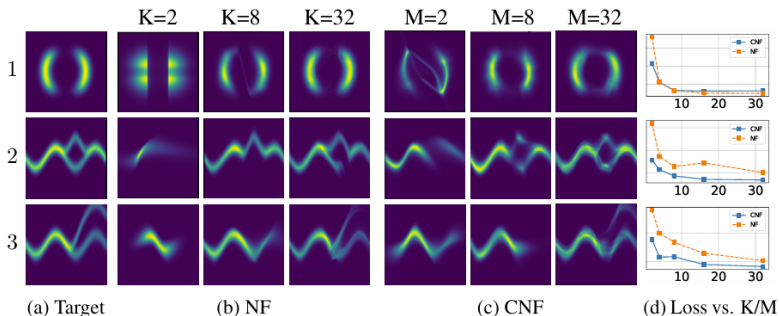
■ Continuous normalizing flow (CNF)

$$\frac{d\mathbf{z}}{dt} = f(\mathbf{z}_t) \Rightarrow \frac{\partial \log p(\mathbf{z}_t)}{\partial t} = -\text{tr} \left(\frac{\partial f}{\partial \mathbf{z}_t} \right)$$

- f no need to be bijective
- Only requires trace operation
- CNF could have multiple hidden units as tr is linear, NF can use layers of only a single hidden unit as $\det$ is nonlinear

$$\frac{d\mathbf{z}}{dt} = \sum_{m=1}^M f_m(\mathbf{z}_t) \Rightarrow \frac{\partial \log p(\mathbf{z}_t)}{\partial t} = - \sum_{m=1}^M \text{tr} \left(\frac{\partial f_m}{\partial \mathbf{z}_t} \right)$$

NF vs. CNF



- K : # of layers

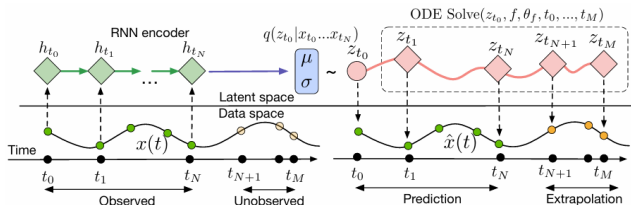
- M : # of hidden units

- **Advantages of CNF**

- CNF with M hidden units is as expressive as NF with $K = M$ layers, but training is more easier for increasing width than increasing depth.
- Reverse transformation for CNF is about the same cost as the forward pass, but not the case for NF.

3. Generate continuous sequential data in latent space

- To deal with missing data in medical records etc.
- **Training algorithm: Variational autoencoder**



- Given observations $x_{t_0}, x_{t_1}, \dots, x_{t_N}$, the RNN encoder infers the parameters for a posterior of z_0

$$q(z_0 | x_{t_0}, \dots, x_{t_N}) = \mathcal{N}(\mu, \sigma)$$

- Sample $z_{t_0} \sim q(z_0 | x_{t_0}, \dots, x_{t_N})$
- Obtain $z_{t_1}, \dots, z_{t_M}$ by $\text{ODESolve}(z_{t_0}, f, t_0, \dots, t_M)$ where f is the Neural ODE parameterized by θ_f and $\frac{\partial \mathbf{z}(t)}{\partial t} = f(\mathbf{z}(t); \theta_f)$
- Map the latent states to data by another neural network: $\hat{x}(t_i) := \hat{x}(t_i, \mathbf{z}(t_i); \theta_x)$
- Maximize the Evidence Lower Bound estimate for $\hat{x}(t_i)$

Spiral reconstruction by Neural ODE

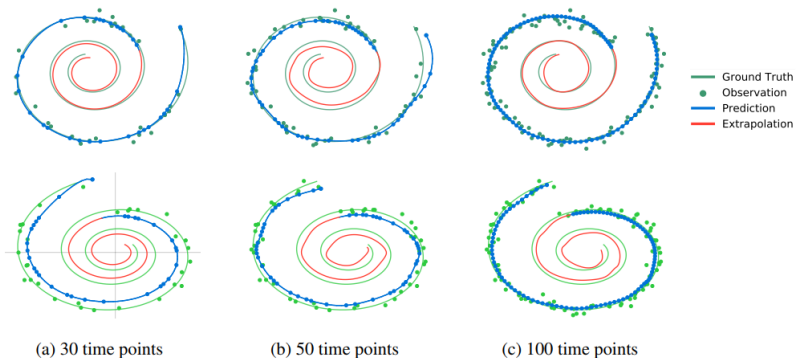


Figure 10: Spiral reconstructions using a latent ODE with a variable number of noisy observations.

4.4 Summary

- We introduced in this chapter the ODEs, and its well-posedness and stability.
- We introduced the one-step methods to solve the ODEs, e.g., the forward Euler method, backward Euler method.
- ResNet is one neural network very similar to numerical ODEs
- Neural ODE is similar to the ODE but with infinite hidden layers. To calculate the gradient, we learned the adjoint method.